

AI-Powered Enterprise E-Commerce Platform

System Architecture Document

Document Information

Item	Description
Project Name	AI-Powered Enterprise E-Commerce Platform
Document Type	System Architecture Document
Version	1.0
Prepared By	Project Team
Status	Draft

Table of Contents

- 1. Introduction
 - 2. Purpose
 - 3. Architectural Goals
 - 4. Architectural Drivers
 - 5. Architecture Style Selection
 - 6. High-Level System Architecture
 - 7. Component Architecture
 - 8. Service Architecture
 - 9. Data Architecture
 - 10. Communication Architecture
 - 11. Security Architecture
 - 12. Deployment Architecture
 - 13. Scalability Considerations
 - 14. Technology Stack
 - 15. Architectural Decisions
 - 16. Future Architecture Enhancements
 - 17. Conclusion
 - 18. References
-

1. Introduction

The System Architecture Document describes the overall structure of the AI-Powered Enterprise E-Commerce Platform.

It defines:

- major system components,
- interactions between services,
- technologies used,
- deployment strategies,
- security mechanisms,
- and scalability considerations.

This document acts as the blueprint for system development.

2. Purpose

The purpose of this document is to:

- define system structure,
 - guide implementation,
 - improve maintainability,
 - support scalability,
 - reduce technical risks.
-

3. Architectural Goals

The architecture should satisfy the following goals:

AG-01 Scalability

Support future growth.

AG-02 Maintainability

Allow modules to be modified independently.

AG-03 Performance

Provide acceptable response times.

AG-04 Security

Protect sensitive customer information.

AG-05 Reliability

Minimize failures and downtime.

AG-06 Extensibility

Allow future AI services to be added easily.

4. Architectural Drivers

The following factors influence architectural decisions.

Functional Drivers

- Product Management
 - Order Processing
 - Recommendations
 - Fraud Detection
 - Forecasting
 - Analytics
-

Non-Functional Drivers

- Security
 - Scalability
 - Availability
 - Performance
 - Maintainability
-

5. Architecture Style Selection

Several architectural styles were evaluated.

Monolithic Architecture

Advantages:

- Simple.
- Easy initial development.

Disadvantages:

- Difficult to scale.

- Difficult to maintain.
- Tight coupling.

Microservices Architecture

Advantages:

- Independent services.
- Better scalability.
- Easier maintenance.
- Technology flexibility.

Disadvantages:

- Higher complexity.
- Service communication challenges.

Selected Architecture

Hybrid Microservices Architecture

The system will use:

- Java Spring Boot Services
- Python AI Services
- Shared Infrastructure Services

This architecture provides flexibility and industry relevance.

6. High-Level System Architecture



7. Architectural Layers

Layer 1 – Presentation Layer

Responsibilities:

- User Interfaces
- Dashboards
- User Interaction

Technologies:

- React (Recommended)
 - HTML/CSS/JavaScript
-

Layer 2 – API Gateway Layer

Responsibilities:

- Route requests
- Authentication validation
- Rate limiting
- Logging

Examples:

- Spring Cloud Gateway
 - NGINX
-

Layer 3 – Business Services Layer

Responsibilities:

- User Management
- Product Management
- Orders
- Payments

Technology:

- Spring Boot
-

Layer 4 – AI Services Layer

Responsibilities:

- Recommendation Engine
- Fraud Detection
- Forecasting
- Customer Segmentation
- AI Assistant

Technology:

- Python FastAPI
-

Layer 5 – Data Layer

Responsibilities:

- Data persistence
- Caching
- Messaging

Technologies:

- PostgreSQL
 - Redis
 - RabbitMQ
-

8. Component Architecture

Authentication Service

Responsibilities:

- Registration
- Login
- JWT generation
- Role management

Technology:

Spring Boot

User Service

Responsibilities:

- Customer profiles
 - Address management
 - User preferences
-

Product Service

Responsibilities:

- Product catalog
 - Categories
 - Product search
-

Inventory Service

Responsibilities:

- Inventory management
 - Stock updates
-

Order Service

Responsibilities:

- Checkout
 - Orders
 - Payments
-

Recommendation Service

Responsibilities:

- Personalized recommendations

Technology:

Python

Fraud Detection Service

Responsibilities:

- Fraud scoring
- Alert generation

Technology:

Python

Forecasting Service

Responsibilities:

- Demand forecasting

Technology:

Python

Notification Service

Responsibilities:

- Emails
 - Internal notifications
-

Analytics Service

Responsibilities:

- Reports
 - Business intelligence
-

9. Service Architecture

Service Communication

Synchronous Communication

Used when immediate responses are required.

Examples:

- Login
- Product Search
- Recommendations

Protocol:

REST APIs

Asynchronous Communication

Used for background tasks.

Examples:

- Notifications
- Analytics updates
- Fraud monitoring

Technology:

RabbitMQ

10. Data Architecture

Primary Database

PostgreSQL

Stores:

- Users
 - Products
 - Orders
 - Payments
 - Inventory
-

Cache Database

Redis

Stores:

- Sessions
 - Frequently accessed products
 - Recommendations cache
-

AI Storage

Stores:

- Training datasets
- Feature stores
- Model artifacts

Data Flow

```
graph TD; A[Customer Actions] --> B[Transactional Database]; B --> C[AI Data Pipeline]; C --> D[Model Training]; D --> E[Predictions];
```

11. Communication Architecture

Java ↔ Python Communication

Communication Method:

REST APIs

Example:

```
graph TD; A[Spring Boot] --> B[POST /recommendations]; B --> C[FastAPI Service]; C --> D[Recommendation Results];
```

Future Communication Enhancements

Possible technologies:

- gRPC
- Apache Kafka
- Event Streaming

12. Security Architecture

Authentication

JWT Authentication.

Authorization

Role-Based Access Control (RBAC).

Communication Security

HTTPS and TLS.

Data Security

- Password hashing
 - Encryption
 - Secure API access
-

Security Layers

```
graph TD; Firewall --> API_Gateway[API Gateway]; API_Gateway --> Authentication_Layer[Authentication Layer]; Authentication_Layer --> Business_Services[Business Services]; Business_Services --> Database_Security[Database Security];
```

Firewall
↓
API Gateway
↓
Authentication Layer
↓
Business Services
↓
Database Security

13. Deployment Architecture

Development Environment

```
graph TD; Developer_Machine[Developer Machine] --> Docker_Containers[Docker Containers];
```

Developer Machine
↓
Docker Containers

Production Environment

```
graph TD; LB[Load Balancer] --> AG[API Gateway]; AG --> SC[Service Cluster]; SC --> DB[Databases];
```

Containerization

Technology:

Docker

Container Orchestration (Future)

Technology:

Kubernetes

14. Scalability Considerations

Horizontal Scaling

Add additional service instances.

Vertical Scaling

Increase hardware resources.

Database Scaling

- Read replicas
 - Partitioning
 - Caching
-

AI Scaling

Deploy independent AI services.

15. Availability and Reliability

Target Availability:

99.9%

Strategies:

- Health checks
 - Monitoring
 - Service redundancy
 - Backups
 - Automatic restart policies
-

16. Monitoring Architecture

Monitoring Tools

Possible technologies:

- Prometheus
 - Grafana
-

Logging Tools

Possible technologies:

- ELK Stack
-

Metrics

Monitor:

- CPU usage
 - Memory usage
 - Response times
 - Error rates
 - Model accuracy
-

17. Technology Stack

Layer	Technology
Frontend	React
API Gateway	NGINX / Spring Gateway
Backend	Spring Boot
AI Services	FastAPI
Database	PostgreSQL
Cache	Redis
Messaging	RabbitMQ
Containerization	Docker
Monitoring	Prometheus
Visualization	Grafana

18. Architectural Decisions Record (ADR)

ADR-001

Decision:

Use Microservices Architecture.

Reason:

Better scalability and maintainability.

ADR-002

Decision:

Use Java and Python together.

Reason:

Leverage Java enterprise capabilities and Python AI ecosystem.

ADR-003

Decision:

Use PostgreSQL.

Reason:

Reliable relational database.

ADR-004

Decision:

Use Docker.

Reason:

Consistent deployment environments.

19. Future Enhancements

Possible future improvements:

- Kubernetes deployment
 - Event-driven architecture
 - Apache Kafka integration
 - Multi-region deployment
 - CI/CD pipelines
 - Service Mesh
 - Generative AI analytics services
-

20. Architecture Risks

Possible risks:

- Service communication complexity.
- Increased deployment complexity.
- Distributed debugging difficulties.
- AI service latency.

Mitigation:

- Documentation
 - Monitoring
 - API standards
 - Containerization
-

21. Conclusion

The proposed architecture provides:

- scalability,
- maintainability,
- flexibility,
- enterprise readiness,
- and strong support for AI integration.

The hybrid microservices architecture enables independent development of business services and AI services while supporting future growth and advanced enterprise capabilities.

References

- [1] Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th Edition). Addison-Wesley.
- [2] Newman, S. (2021). Building Microservices (2nd Edition). O'Reilly.
- [3] Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture.
- [4] Sommerville, I. (2016). Software Engineering (10th Edition). Pearson.
- [5] Fowler, M. (2018). Patterns of Enterprise Application Architecture.
- [6] NGINX Documentation.
- [7] Docker Documentation.
- [8] FastAPI Documentation.
- [9] Spring Boot Documentation.